

Efficient and Probabilistic Adaptive Voxel Mapping for Accurate Online LiDAR Odometry

Chongjian Yuan^{1,2}, *Graduate Student Member, IEEE*, Wei Xu¹, *Graduate Student Member, IEEE*, Xiyuan Liu¹, *Graduate Student Member, IEEE*, Xiaoping Hong², *Member, IEEE*, and Fu Zhang,¹ *Member, IEEE*

Abstract—This paper proposes an efficient and probabilistic adaptive voxel mapping method for LiDAR odometry. The map is a collection of voxels; each contains one plane feature that enables the probabilistic representation of the environment and accurate registration of a new LiDAR scan. We further analyze the need for coarse-to-fine voxel mapping and then use a novel voxel map organized by a Hash table and octrees to build and update the map efficiently. We apply the proposed voxel map to an iterated extended Kalman filter and construct a maximum a posteriori probability problem for pose estimation. Experiments on the open KITTI dataset show the high accuracy and efficiency of our method compared to other state-of-the-art methods. Experiments on indoor and unstructured outdoor environments with solid-state LiDAR and non-repetitive scanning LiDAR further verify the adaptability of our mapping method to different environments and LiDAR scanning patterns (see our attached video¹). Our codes and dataset are open-sourced on Github²

Index Terms—Mapping; Localization; SLAM

I. INTRODUCTION

IN recent years, with the development of 3D LiDAR technology, especially the emergence of low-cost high-density solid-state LiDARs [1], LiDAR(-inertial) odometry has been increasingly used in various applications such as autonomous vehicles [2], UAVs (Unmanned Aerial Vehicles) [3], [4] and 3D mobile mapping devices [5].

A fundamental requirement of LiDAR odometry is a proper representation of the environment (i.e., map) that can be maintained efficiently and can register new points effectively. A predominant mapping method in existing LiDAR(-inertial) odometry, such as LOAM [6], Lego-LOAM [7], LINS [8], LIO-SAM [9], and Lili-OM [10], is a point cloud map, which consists of raw or selected edge or plane points. When registering a new scan, each point in the scan is registered to a small plane fitted by a few nearest points in the map by iterated closet point (ICP) [11] or its variants (e.g., generalized ICP [12]). This line of work consummates in FAST-LIO2 [13], which develops an incremental kd-tree structure to organize the point cloud map efficiently.

Manuscript received: February 24, 2022; Revised: May 17, 2022; Accepted: June 11, 2022.

This paper was recommended for publication by Editor Javier Civera upon evaluation of the Associate Editor and Reviewers' comments. (Corresponding authors: Fu Zhang; Xiaoping Hong)

¹C. Yuan, W. Xu, X. Liu and F. Zhang are with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong Special Administrative Region, People's Republic of China. {ycj1, xuwei1, xliu1, fuzhang}@connect.hku.hk

²C. Yuan and X. Hong are with the School of System Design and Intelligent Manufacturing, Southern University of Science and Technology, Shenzhen, People's Republic of China. {yuancj2020, hongxp}@sustech.edu.cn

Digital Object Identifier (DOI): see top of this page.

¹<https://youtu.be/HSwQdXg31WM>

²<https://github.com/hku-mars/VoxelMap>

As a direct form of LiDAR measurements, a point cloud map is simple to implement. However, a major drawback of the point cloud map is difficult to consider map uncertainty caused by LiDAR measurement noise. In ICP or its variants, the plane fitted by the nearest map points is usually regarded as the true deterministic plane that a new point lies on, while it should really be probabilistic due to measurements noises in the map points. While incorporating these noises in the fitted plane (and the subsequent point registration) is possible, it is less reliable (since the nearest points are usually few in order to avoid spurious planes) and time-consuming (since the uncertainty has to be recomputed if any nearest point is changed during the registration).

Consideration of the map uncertainty requires explicit parameterization of salient features (e.g., plane) in the environment, tracking these features across different LiDAR scans and estimating these feature parameters and their uncertainties. These tasks are challenging due to the various possible environments which contain features of different sizes. The problem is further complicated by the coarse-to-fine phenomenon widely in LiDAR measurements. For example, points in a scan or map usually have drastically different densities in the space due to varying LiDAR resolution, scanning types, and considerably uneven distribution of points in the space. The point density also varies over time due to the sequentially scanning nature of LiDAR sensors.

To address the above challenges, this paper proposes a novel online adaptive voxel mapping method, which constructs voxels of different sizes to adapt to the variation in environment structures and point density. A voxel tracks one salient feature (we use plane in this paper). The feature parameters and uncertainties are repeatedly estimated within the voxel based on received points. The parameterized feature allows us to consider the uncertainty of both the current point and the map accurately. Concretely, the contributions of this paper include:

- 1) An adaptive-size and coarse-to-fine voxel construction method, which is adaptable to environments of varying structures and robust to the sparsity and irregularity of LiDAR point cloud. The adaptive voxel map is organized in an octree-hash data structure to increase the efficiency of voxel construction, updates, and inquiries.
- 2) A true probabilistic map representation, where each feature (i.e., plane) contained in the voxel map accurately considers uncertainties arising from both point measurement noise and pose estimation error.
- 3) An open-source implementation of the proposed mapping method in a LiDAR(-inertial) odometry system and full validation of the design on real-world datasets of various environments (structured and unstructured) and LiDARs (multi-spinning LiDARs and non-conventional

solid-state LiDARs). In particular, the superiority of our method over other state-of-art methods on KITTI dataset is demonstrated.

II. RELATED WORKS

On the one hand, the plane feature used in our voxel map is most similar to the surfel used in Suma [14] and Suma++ [15], which also register new points by optimizing the point to plane residuals and explicitly maintain and estimate the surfel in the map. **The surfel in [14], [15] is a very small planar feature, which is also the case of our plane features in cluttered environments.** However, due to the adaptive sizes in our voxel maps, our plane features could be very large in structured environments. This adaptive strategy makes our method suitable for environments with varying structures [16] while ensuring high computation efficiency. Another difference between our plane features with the surfels in [14] and [15] lies in the surfel/plane association. In [14], [15], both surfels in the map and new LiDAR points are projected (i.e., rendered) to a virtual image plane to perform point-to-plane association. Such a surfel rendering process is very time-consuming and typically requires GPU acceleration for online applications. In contrast, our plane association is made by efficient Hash table inquiry.

The computation issue in the surfel rendering could be mitigated by modeling the surfel map as a Gaussian mixture model and performing surfel association by Expectation Maximization, as shown in MRSLaserMap [17] and [18]. The works in [17], [18] also adopted a multi-resolution surfel map, where the valid surfels are merged from the finest resolution to coarser resolution. This fine-to-coarse map method originates from MRSMMap [19], which was inspired from the radial nature of the LiDAR scanner, and is usually sensitive to the sparsity and irregularity of the LiDAR point cloud. In contrast, our method constructs voxel maps in a coarse-to-fine manner and is more robust to LiDAR point cloud's irregularity and adaptable to different environments.

Instead of deterministic surfels, the works in [20], [21] incorporate LiDAR measurements noises into surfels, leading to a probabilistic point-to-surfel registration similar to our probabilistic plane representation. However, [20], [21] only considered the LiDAR measurement noises but ignored the uncertainties caused by errors in LiDAR pose estimation. In contrast, our plane uncertainty considers both uncertainties. Notice that the pose estimate is used to transform LiDAR points measured in the local body frame to the world frame where the surfel/plane is estimated, hence the pose estimation error will contribute to the surfel/plane uncertainty.

On the other hand, our voxel map is similar to the normal distribution transformation (NDT) [22], where the map consists of many voxels (of different sizes, e.g., [23]). In each voxel, a 3D Gaussian distribution is fitted from map points lying in the voxel and is used to register a new point by maximizing the likelihood of the point being taken from the distribution. LiTAMIN2 [24] further improves the accuracy of NDT by incorporating ICP with NDT. When compared with these NDT methods, our method explicitly parameterizes and estimates a plane in a voxel. Such explicit

plane parameterization allows performing direct point-to-plane registration, which is more sound and often accurate than NDT where the 3D Gaussian distribution over constrains the new point by penalizing all three directions (instead of the plane normal only). The explicit plane parameterization also allows incorporating noises in both LiDAR point measurements and pose estimation, which provides further accuracy improvements.

In terms of data structure, our proposed octo-hash data structure and coarse-to-fine mapping method are similar to OGN [25] and HSP [26]. OGN [25] introduces an octo-hash representation into a deep convolution decoder, and achieves computation- and memory-efficient generation of volumetric 3D output. HSP [26] uses a coarse-to-fine prediction on a hierarchical surface for a more accurate 3D object reconstruction. While good results of 3D object output and object reconstruction from OGN [25] and HSP [26] are presented, we integrate the efficient octo-hash data structure and coarse-to-fine plane representation into a new type of map, and apply the map to the LiDAR odometry problem, enhancing the adaptability, efficiency and accuracy of LiDAR odometry.

III. METHODOLOGY

A. Probabilistic Plane Representation

Our voxel map contains one probabilistic feature in each voxel. Without loss of generality, we use plane features due to their vast availability in environments and present the uncertainty model of a plane feature in this section.

Since a plane feature is estimated from its associated points, any noises in the points will contribute to the plane estimation uncertainty. We first investigate the point noises in Section III-A1 and then derive how the noises contribute to the plane uncertainty in Section III-A2. Notice that since the voxel map (hence the contained plane features) is represented in the world frame, the point noises should also be investigated in the world frame, which leads to two possible noise sources: one is the raw point measurement noises, which is relative to the local LiDAR body frame, and the other is the error in LiDAR pose estimation that projects the local LiDAR points to the world frame.

1) Uncertainty of point ${}^w\mathbf{p}_i$:

According to the analysis of the measurement noises for LiDAR sensors in [27], the uncertainty of a LiDAR point in the local LiDAR frame consists of two parts, the ranging uncertainty and the bearing direction uncertainty (see Fig. 1 (a)). Let $\omega_i \in \mathbb{S}^2$ be the measured bearing direction, $\delta_{\omega_i} \sim \mathcal{N}(\mathbf{0}_{2 \times 1}, \Sigma_{\omega_i})$ be the bearing direction noise in the tangent plane of ω_i , d_i be the depth measurement and $\delta_{d_i} \sim \mathcal{N}(0, \Sigma_{d_i})$ be the ranging noise. Then the noise $\delta_{L\mathbf{p}_i}$ of the measured point ${}^L\mathbf{p}_i$ and its covariance $\Sigma_{L\mathbf{p}_i}$ is

$$\begin{aligned} \delta_{L\mathbf{p}_i} &= \underbrace{\begin{bmatrix} \omega_i & -d_i[\omega_i]_{\wedge} \mathbf{N}(\omega_i) \end{bmatrix}}_{\mathbf{A}_i} \begin{bmatrix} \delta_{d_i} \\ \delta_{\omega_i} \end{bmatrix} \sim \mathcal{N}(\mathbf{0}, \Sigma_{L\mathbf{p}_i}), \\ \Sigma_{L\mathbf{p}_i} &= \mathbf{A}_i \begin{bmatrix} \Sigma_{d_i} & \mathbf{0}_{1 \times 2} \\ \mathbf{0}_{2 \times 1} & \Sigma_{\omega_i} \end{bmatrix} \mathbf{A}_i^T. \end{aligned} \quad (1)$$

where $\mathbf{N}(\omega_i) = [\mathbf{N}_1 \ \mathbf{N}_2] \in \mathbb{R}^{3 \times 2}$ is an orthonormal basis of the tangent plane at ω_i , $[\]_{\wedge}$ denotes the skew-symmetric

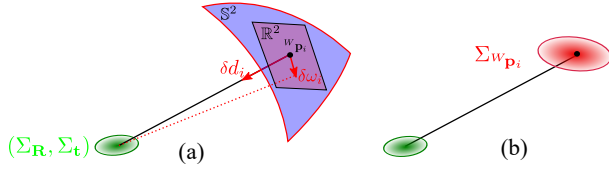


Fig. 1: The uncertainty model of the registered point: (a) the LiDAR measurement uncertainty model, including bearing direction and depth measurement uncertainty; (b) point covariance Σ_{Wp_i} computed from (3).

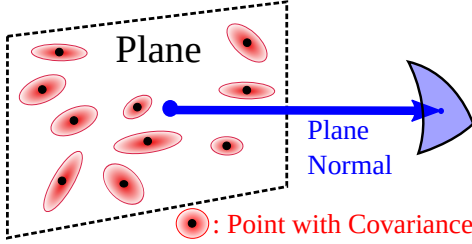


Fig. 2: The Uncertainty model of the plane normal.

matrix mapping the cross product. Detailed derivation of equation (1) can be found in [27].

Considering that the LiDAR point ${}^L p_i$ will be further projected to the world frame through the estimated pose ${}^W \tilde{T} = ({}^W R, {}^W t) \in SE(3)$, with estimation uncertainty (Σ_R, Σ_t) as shown in Fig.1 (a), by the following rigid transformation

$${}^W p_i = {}^W R {}^L p_i + {}^W t \quad (2)$$

Therefore, the uncertainty of the LiDAR point ${}^W p_i$ is hence

$$\Sigma_{Wp_i} = {}^W R \Sigma_{Lp_i} {}^W R^T + {}^W R [{}^L p_i]_{\wedge} \Sigma_R [{}^L p_i]_{\wedge}^T {}^W R^T + \Sigma_t \quad (3)$$

where Σ_R is the uncertainty of the ${}^W R$ in the tangent space and Σ_t is the uncertainty of the ${}^W t$. Considering the uncertainty due to LiDAR measurement and pose estimation, the covariance of point cloud at different positions has a great difference: points at closer distances have their noises dominated by the ranging noise while at further distances dominated by the bearing noise. The uncertainty analysis of a single LiDAR point is also the basis of the uncertainty model of a plane feature.

2) Plane Uncertainty Modelling:

Let a plane feature consist of a group of LiDAR points ${}^W p_i$ ($i = 1, \dots, N$), each has an uncertainty Σ_{Wp_i} due to the measurement noise and pose estimation error as shown in (3). Denote the points covariance matrix be A :

$$\bar{p} = \frac{1}{N} \sum_{i=1}^N {}^W p_i; \quad A = \frac{1}{N} \sum_{i=1}^N ({}^W p_i - \bar{p})({}^W p_i - \bar{p})^T; \quad (4)$$

Then, the plane can be represented by its normal vector $\mathbf{n}$, which is the eigenvector associated with the minimum eigenvalue of A , and the point $\mathbf{q} = \bar{p}$, which lies in this plane. As both A and $\bar{p}$ are dependent on ${}^W p_i$, we can denote the plane parameters $(\mathbf{n}, \mathbf{q})$ as functions of ${}^W p_i$ shown below:

$$[\mathbf{n}, \mathbf{q}]^T = \mathbf{f}({}^W p_1, {}^W p_2, \dots, {}^W p_N). \quad (5)$$

Based on the uncertainty analysis in III-A1, each LiDAR point ${}^W p_i$ has a noise $\delta_{Wp_i} \sim \mathcal{N}(\mathbf{0}_{3 \times 1}, \Sigma_{Wp_i})$. Therefore, the

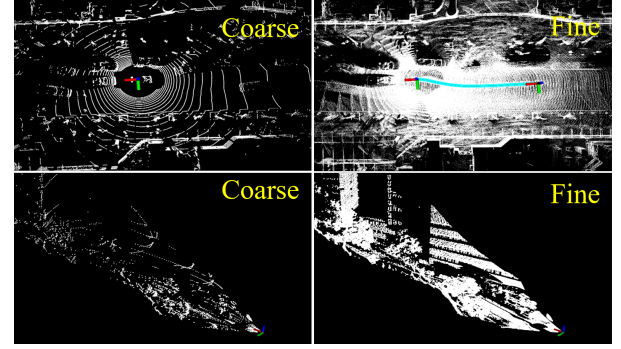


Fig. 3: The coarse-to-fine phenomena in a spinning LiDAR (top two figures) and a non-repetitive solid-state LiDAR (bottom two figures).

ground-truth normal vector $\mathbf{n}^{gt}$ and ground-truth position $\mathbf{q}^{gt}$ (the real value in the real world that is to be estimated) are

$$\begin{aligned} [\mathbf{n}^{gt}, \mathbf{q}^{gt}]^T &= \mathbf{f}({}^W p_1 + \delta_{Wp_1}, {}^W p_2 + \delta_{Wp_2}, \dots, {}^W p_N + \delta_{Wp_N}) \\ &\approx [\mathbf{n}, \mathbf{q}]^T + \sum_{i=1}^N \frac{\partial \mathbf{f}}{\partial {}^W p_i} \delta_{Wp_i} \end{aligned} \quad (6)$$

Here, $\frac{\partial \mathbf{f}}{\partial {}^W p_i} = [\frac{\partial \mathbf{n}}{\partial {}^W p_i}, \frac{\partial \mathbf{q}}{\partial {}^W p_i}]^T$. Assume A has the eigenvector matrix U , the minimum eigenvalue λ_3 and the corresponding eigenvector $\mathbf{u}_3$, referring to [28], we can take the derivative of the $\mathbf{n}$ and $\mathbf{q}$ with respect to each point ${}^W p_i$ as below:

$$\begin{aligned} \frac{\partial \mathbf{n}}{\partial {}^W p_i} &= U \begin{bmatrix} \mathbf{F}_{1,3}^{p_i} \\ \mathbf{F}_{2,3}^{p_i} \\ \mathbf{F}_{3,3}^{p_i} \end{bmatrix}, \quad \mathbf{F}_{m,3}^{p_i} = \begin{cases} \frac{({}^W p_i - \mathbf{q})^T (\mathbf{u}_m \mathbf{n}^T + \mathbf{n} \mathbf{u}_m^T)}{N(\lambda_3 - \lambda_m)}, & m \neq 3, \\ \mathbf{0}_{1 \times 3}, & m = 3. \end{cases} \\ \frac{\partial \mathbf{q}}{\partial {}^W p_i} &= \text{diag} \left(\frac{1}{N}, \frac{1}{N}, \frac{1}{N} \right) \end{aligned} \quad (7)$$

Then covariance matrix $\Sigma_{\mathbf{n}, \mathbf{q}}$ of $\mathbf{n}$ and $\mathbf{q}$ is therefore:

$$\Sigma_{\mathbf{n}, \mathbf{q}} = \sum_{i=1}^N \frac{\partial \mathbf{f}}{\partial {}^W p_i} \Sigma_{Wp_i} \frac{\partial \mathbf{f}}{\partial {}^W p_i}^T \quad (8)$$

It is seen that $\mathbf{n}$ and $\mathbf{q}$ are not independent as they are calculated from the same set of noisy points.

B. Coarse-to-fine and Efficient Voxel Map Construction

In this section, we first explain the motivation for a coarse-to-fine voxel-based map and then the methodology to build and update the voxel map efficiently.

1) Motivation:

As shown in Fig. 3, LiDAR points are typically sampled sequentially, hence a scan of the point cloud is always accumulated from sparse to dense, especially in outdoor environments where the points are distributed in a larger space. When the point cloud is relatively sparse, common surfel-based fine-to-coarse mapping methods can usually obtain a very small number of planes only, limiting their applications to high-resolution LiDARs and relatively low scan rate (e.g., 10Hz) such that a sufficient number of points could be accumulated. This would, however, require compensating the motion within the accumulated period. To address this issue, we propose a coarse-to-fine voxel mapping method that can build a rough voxel map when the point cloud is sparse and refine the map when more points are received.

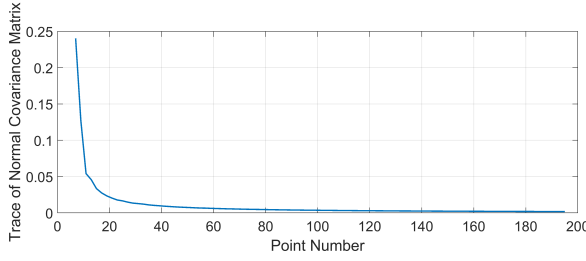


Fig. 4: The convergence of the trace of the normal vector covariance matrix with the increase of the number of points.

2) Voxel Map Construction :

To achieve voxel map construction in a coarse-to-fine manner, we build an adaptive voxel map organized by a Hash table and an octree for each Hash entry. More specifically, we first **cut the space (in the global world frame) into voxels, each with the size of the coarse map resolution**. Then, for the first LiDAR scan, which defines the world frame, the contained points are distributed into voxels. **Voxels populated with points are indexed into a Hash table**. Then, for each populated voxel, if all the contained points lie on a plane (the minimum eigenvalue of the point covariance matrix is less than a specified threshold), we store the plane points and calculate the plane parameters $(\mathbf{n}, \mathbf{q})$ as in (5) and their uncertainty $\Sigma_{\mathbf{n}, \mathbf{q}}$ as in (8); otherwise, the current voxel will break into eight octants and repeat plane checking and voxel cutting in each one until reaching the maximum number of layers. Notice that the voxels have different sizes, each voxel contains one plane feature fitted from the contained LiDAR raw points.

3) Voxel Map Update:

For online LiDAR odometry, new scans of LiDAR point cloud are continuously coming and their poses are estimated as in Section III-D. Then the estimated pose is used to register the new points into the global map. **When the new points lie in an unpopulated voxel**, it will construct the voxel. Otherwise, **when the new points are added to an existing voxel**, the parameters and the uncertainty of the plane in the voxel should be updated. This will cause a growing processing time with the constant reception of new points. To address this issue, we find that the uncertainty of the plane parameters will quickly converge as illustrated in Fig. 4, where the position of each point carries a Gaussian noise with a zero mean and a variance of $0.1m^2$. It is seen that the uncertainty of the normal vector converges when the number of points reaches 50. After the uncertainty converges, we discard all historical points and retain the estimated plane parameters $(\mathbf{n}, \mathbf{q})$ and covariance $\Sigma_{\mathbf{n}, \mathbf{q}}$. Once more new points are coming, we keep only the latest 10 points and calculate the new plane normal vector composed of those 10 points. If the new normal vector and the previously converged normal vector continue to appear a relatively large difference, we assume that this area of the map has changed and needs to be reconstructed as Section III-B2.

C. Point-to-plane Match

This section details how to match points in a new LiDAR scan with the voxel map to construct the constrain for the pose estimation and subsequent point cloud registration. Based on

the accurate uncertainty modeling of the points and planes, we could easily implement the point-to-plane scan match. Given a LiDAR point ${}^W\mathbf{P}_i$ predicted in the world frame with the pose prior, we first find which **root voxel** (with the coarse map resolution) it lies in by its Hash key. Then, all the contained sub-voxels are polled for a possible match with the point. Specifically, let a sub-voxel contains a plane with normal $\mathbf{n}_i$ and center $\mathbf{q}_i$, we calculate the point-to-plane distance:

$$d_i = \mathbf{n}_i^T ({}^W\mathbf{P}_i - \mathbf{q}_i) \quad (9)$$

As analyzed above, the normal vector $\mathbf{n}_i$, the LiDAR point ${}^W\mathbf{P}_i$, and the center $\mathbf{q}_i$ have uncertainties. Considering all these uncertainties, we obtain:

$$\begin{aligned} d_i &= (\mathbf{n}_i^{gt} \boxplus \delta_{\mathbf{n}_i})^T [({}^W\mathbf{P}_i^{gt} + \delta_{W\mathbf{P}_i}) - \mathbf{q}_i^{gt} - \delta_{\mathbf{q}_i}] \\ &\approx \underbrace{\mathbf{n}_i^{gtT} ({}^W\mathbf{P}_i^{gt} - \mathbf{q}_i^{gt})}_0 + \underbrace{\mathbf{J}_{\mathbf{n}_i} \delta_{\mathbf{n}_i} + \mathbf{J}_{\mathbf{q}_i} \delta_{\mathbf{q}_i} + \mathbf{J}_{W\mathbf{P}_i} \delta_{W\mathbf{P}_i}}_{\mathbf{w}_i} \end{aligned}$$

which implies

$$d_i \sim \mathcal{N}(0, \Sigma_{\mathbf{w}_i}), \quad \Sigma_{\mathbf{w}_i} = \mathbf{J}_{\mathbf{w}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, W\mathbf{P}_i} \mathbf{J}_{\mathbf{w}_i}^T, \quad (10)$$

where,

$$\begin{aligned} \mathbf{J}_{\mathbf{w}_i} &= [\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{\mathbf{q}_i}, \mathbf{J}_{W\mathbf{P}_i}] = [({}^W\mathbf{P}_i - \mathbf{q}_i)^T, -\mathbf{n}_i^T, \mathbf{n}_i^T] \\ \Sigma_{\mathbf{n}_i, \mathbf{q}_i, W\mathbf{P}_i} &= \begin{bmatrix} \Sigma_{\mathbf{n}_i, \mathbf{q}_i} & 0 \\ 0 & \Sigma_{W\mathbf{P}_i} \end{bmatrix}. \end{aligned} \quad (11)$$

That is being said, if the point lies on the candidate plane, its distance d_i should be subject to the distribution in (10). Therefore, let the standard deviation of the distribution be $\sigma = \sqrt{\Sigma_{\mathbf{w}_i}}$, we can examine whether the measured point-to-plane distance falls within 3σ . If so, it is selected to be an effective **match**. Besides, if a point matches more than one plane based on the 3σ criterion, the plane with the highest probability will be matched. If no plane passes the 3σ test, the point is discarded to remove the possible false matches caused by voxel quantization.

D. State Estimation

We build a LiDAR(-inertial) odometry system based on an iterated extended Kalman filter similar to FAST-LIO2 [13]. Assume that we are given a state estimation prior $\hat{\mathbf{x}}_k$ with covariance $\hat{\mathbf{P}}_k$. The prior can be provided from a constant velocity assumption for LiDAR-only Odometry (experiment A and B) or IMU propagation for LiDAR-inertial Odometry (experiment C). This prior will be fused with the point-to-plane distance matched in Section III-C to form a maximum a posteriori (MAP) estimation. Specifically, the i -th **valid point-to-plane match** leads to the observation equation

$$\mathbf{z}_i = \mathbf{h}_i(\mathbf{x}_k) + \mathbf{v}_i \quad (12)$$

where $\mathbf{z}_i$ is the point-to-plane distance residual d_i in equation (9), $\mathbf{h}_i(\mathbf{x}_k)$ is the observation function and $\mathbf{v}_i \sim (0, \mathbf{R}_i)$ is the observation noise. By substituting the state $\mathbf{x}_k$ (i.e., the sensor pose $\mathbf{T}_k$) into (9) and linearize it around the current state update $\hat{\mathbf{x}}_k$, we obtain:

$$\begin{aligned} \mathbf{z}_i &= \mathbf{h}_i(\mathbf{x}_k) + \mathbf{v}_i = \mathbf{n}_i^T ({}^W\mathbf{P}_i - \mathbf{q}_i) \\ &= (\mathbf{n}_i^{gt} \boxplus \delta_{\mathbf{n}_i})^T [(\mathbf{T}_k^{gt} \boxplus \delta_{\mathbf{T}_k}) ({}^L\mathbf{P}_i^{gt} + \delta_{L\mathbf{P}_i}) - \mathbf{q}_i^{gt} - \delta_{\mathbf{q}_i}] \\ &\approx \underbrace{\mathbf{n}_i^{gtT} (\mathbf{T}_k^{gt} {}^L\mathbf{P}_i^{gt} - \mathbf{q}_i^{gt})}_0 + \underbrace{\mathbf{J}_{\mathbf{T}_k} \delta_{\mathbf{T}_k}}_{\mathbf{H}_i \delta \mathbf{x}_k} + \underbrace{\mathbf{J}_{\mathbf{n}_i} \delta_{\mathbf{n}_i} + \mathbf{J}_{\mathbf{q}_i} \delta_{\mathbf{q}_i} + \mathbf{J}_{L\mathbf{P}_i} \delta_{L\mathbf{P}_i}}_{\mathbf{v}_i} \end{aligned}$$

which implies

$$\mathbf{R}_i = \mathbf{J}_{\mathbf{v}_i} \Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{p}_i} \mathbf{J}_{\mathbf{v}_i}^T, \quad (13)$$

where

$$\mathbf{J}_{\mathbf{v}_i} = [\mathbf{J}_{\mathbf{n}_i}, \mathbf{J}_{\mathbf{q}_i}, \mathbf{J}_{\mathbf{p}_i}] = [(\bar{\mathbf{T}}_k^L \mathbf{p}_i - \mathbf{q}_i)^T, -\mathbf{n}_i^T, \mathbf{n}_i^T \bar{\mathbf{R}}_k] \\ \Sigma_{\mathbf{n}_i, \mathbf{q}_i, \mathbf{p}_i} = \begin{bmatrix} \Sigma_{\mathbf{n}_i, \mathbf{q}_i} & 0 \\ 0 & \Sigma_{\mathbf{p}_i} \end{bmatrix}.$$

Finally, combining the state prior with all effective measurements, we can obtain the MAP estimation:

$$\min_{\mathbf{x}_k} \left(\|\mathbf{x}_k \boxminus \hat{\mathbf{x}}_k\|_{\hat{\mathbf{P}}_k}^2 + \sum_{i=1}^m \|d_i - \mathbf{H}_i \cdot (\mathbf{x}_k \boxminus \bar{\mathbf{x}}_k)\|_{\bar{\mathbf{R}}_i}^2 \right) \quad (14)$$

where the first part is the state prior and the second part is the measurement observation.

IV. EXPERIMENTS

In this section, in order to show the high accuracy, efficiency, and adaptability of our mapping method to different environments and LiDAR types, we conduct experiments on three typical LiDARs in three different environments (urban environment, indoor environment, and unstructured outdoor environment), respectively. The details of the LiDAR we used are shown in Table I. In each experiment, we compare our method with state-of-the-art counterparts. All the experiments are conducted on a desktop computer with Intel i7-10700 @ 2.9 GHz with 16 GB RAM and Nvidia GeForce GTX 730 with 2 GB RAM (in case a GPU is used).

Sensor	Type	Frequency	Scan pattern	FoV
HDL-64E S2	Mechanical	10Hz	repetitive	$360^\circ \times 32^\circ$
Realsense L515	Solid-state	30Hz	repetitive	$70^\circ \times 55^\circ$
Livox Avia	Solid-state	10Hz	non-repetitive	$70^\circ \times 77^\circ$

TABLE I: Different Types of LiDAR

A. Urban Environment Test

In this experiment, we evaluate our approach on the odometry datasets of the KITTI Vision Benchmark [2], where the LiDAR data is collected by Velodyne HDL-64E S2 in urban environments and the in-frame motion has been compensated in advance. As in [29], a vertical angle of 0.22 degree is employed to correct the calibration errors in raw point clouds from the KITTI dataset.

Comparison with state-of-the-art. We compare our method with FAST-LIO2 [13], a point-cloud map-based LiDAR-inertial odometry approach, MULLS [30], a multi-feature-based LO system, LiTAMIN2 [24], an improved ICP-NDT hybrid method, Suma [14], a surfel-based LiDAR SLAM approach, and Lego-Loam [7], a ground-optimized LiDAR odometry and mapping system.

For the implementation, we run our algorithm on all sequences with the same parameters, where the maximum voxel size is 3m and the maximum octo-tree layer is 3, leading to a minimum voxel size of 0.375m. The full version of the algorithm is denoted as *Ours (full)*. To verify the effectiveness of the adaptive voxel module and features probabilistic module, we use a fixed size voxel (i.e., 2m) (termed as *Ours (w/o adaptive)*) and a zero plane uncertainty in (8) (termed

as *Ours (w/o prob.)*), respectively, while retaining all other modules with the same parameter values. For FAST-LIO2, it is designed for LiDAR-inertial fusion, where IMU propagation is served as a motion prior. Since the KITTI dataset does not have IMU data, we use a constant velocity model similar to ours (see (14)) to serve as the motion prior in FAST-LIO2 for a fair comparison. The other parameters of FAST-LIO2 are their default values recommended for multi-line spinning LiDARs in outdoor environments. For MULLS, we conduct the experiments using the open-sourced implementation with the default parameters. For LiTAMIN2, we directly use the results presented in the original paper [24]. The work in LiTAMIN2 [24] further benchmarked their results with Suma [14] and Lego-Loam [7], both with loop closure and without loop closure. We directly draw these results and compare them with ours. Since our implementation is a LiDAR odometry without any loop closure, for the sake of fair comparison, we only compare with the results where loop closure for MULLS, LiTAMIN2, Suma, and Lego-Loam are off.

Results and Discussion. Table II shows the absolute trajectory error (ATE) [31] of all methods, where the results for LiTAMIN2, Suma, and Lego-Loam are directly drawn from [24]. As can be seen, our method (*Ours (full)*) has the best overall accuracy than all other algorithms, especially in long-term sequences like 00, 02 and 08. To further show the high accuracy of our method, we plot the estimated LiDAR trajectories of all sequences and their ground-truth in Fig. 5. It is seen that all of our global trajectories are very close to the ground truth, verifying the high accuracy of our method. Notice that all variants of our methods use the same parameters across all sequences.

The improvement of our is mainly attributed to the adaptive voxelization and probabilistic plane representation of the map. The effectiveness of the adaptive voxelization can be seen from the performance degradation from *Ours (full)* to *Ours (w/o adaptive)*. With the adaptive voxelization, our method **is able to capture plane features of different sizes** (both large-size and finer-level small planes) in the environment. The effectiveness of the probabilistic plane representation can be seen from the performance degradation from *Ours (full)* to *Ours (w/o prob.)*. Without the probabilistic plane representation, the **performance** of our method is at the same level of FAST-LIO2, which is reasonable because FAST-LIO2 also uses **a deterministic plane representation** and the same IKFoM framework [32]. Compared to the adaptive voxelization, the probabilistic plane representation plays a much more significant role in the accuracy since the performance degradation by turning off the module is much larger. Finally, it is also interesting to see that our method with only probabilistic plane representation *Ours (w/o adaptive)* can even exceed all comparison methods in most sequences.

Fig. 6 illustrates the plane features estimated in our voxel maps. As can be seen, our voxel map contains plane features of different sizes: planes on the ground and walls are usually large while those on cluttered objects (e.g., moving cars, roadside objects) and their nearby regions are small. Plane features contained in the voxel map are also colored by their respective uncertainty (the trace of the plane covariance matrix (8) in Fig.

TABLE II: Accuracy (ATE in meters) Comparison on KITTI Odometry Training Sequences

Approach	00	01	02	03	04	05	06	07	08	09	10	Avg. of all scans
(Total length [m])	(3724)	(2453)	(5067)	(560)	(393)	(2205)	(1232)	(694)	(3222)	(1705)	(919)	[deg]/[m]
Ours (full)	0.9/2.8	1.9/7.8	1.7/6.1	1.2/0.7	0.6/0.3	0.8/1.2	0.4/0.4	0.7/0.7	1.1/2.3	1.0/1.9	1.0/1.1	1.2/2.9
Ours (w/o adaptive)	1.1/3.1	2.0/8.2	1.7/7.1	1.2/0.7	0.7/0.5	0.8/1.3	0.5/0.5	0.9/0.7	1.8/3.1	1.1/2.0	1.3/1.2	1.3/3.4
Ours (w/o prob.)	2.2/3.9	2.5/9.5	1.9/8.1	1.4/0.9	0.7/0.4	1.0/1.2	0.5/0.6	1.0/0.6	3.0/6.8	1.2/1.9	1.2/1.5	1.8/4.5
LiTAMIN2 [24]	1.6/5.8	3.5/15.9	2.7/10.7	2.6/0.8	2.3/0.7	1.1/2.4	1.1/0.9	1.0/0.6	1.3/2.5	1.7/2.1	1.2/1.0	1.8/5.1
MULLS [30]	1.7/6.1	1.0/2.4	2.4/10.6	0.7/0.7	0.2/0.9	1.0/2.4	0.4/0.6	0.5/0.6	1.9/4.3	1.4/2.5	0.5/1.1	1.5/4.8
Suma [14]	1.0/2.9	3.2/13.8	2.2/8.4	1.5/0.9	1.8/0.4	0.7/1.2	0.4/0.4	0.7/0.5	1.5/2.8	1.1/2.9	0.8/1.3	1.4/3.9
FAST-LIO2 [13]	2.0/3.8	2.5/11.9	2.8/12.8	1.2/0.8	0.7/0.5	1.2/2.1	1.1/1.2	1.2/0.8	1.8/3.0	1.4/2.0	1.1/1.5	1.8/4.9
Lego-Loam [7]	2.8/6.3	3.8/119.4	4.1/14.7	4.1/0.9	3.3/0.8	1.9/2.8	1.4/0.8	1.5/0.7	2.5/3.5	2.2/2.1	1.9/1.8	2.8/11.1

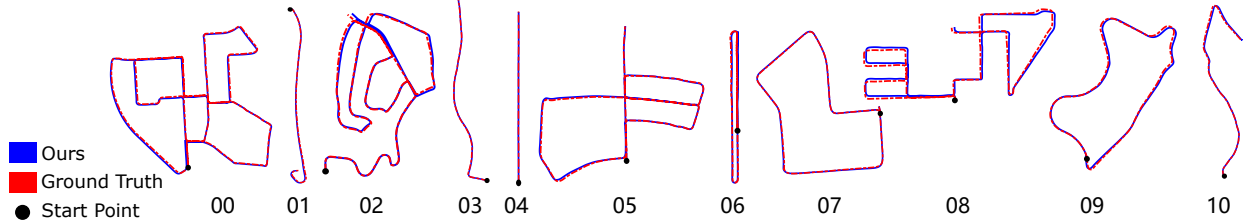


Fig. 5: Trajectories of sequence 00 to 10. The blue trajectories are the camera paths obtained from our method and the red trajectories are the ground truth. The black dot represents the starting point.

6. As can be seen, plane features far away from the road have much higher uncertainty than that nearby. This is caused by the uneven LiDAR point measurements, where nearby areas have many more measurements than far areas due to the repeat observation. We also notice that plane features in cluttered areas have higher uncertainties than that in structured areas (e.g., ground, wall) and that plane features on moving objects (e.g., cars) have higher uncertainties than static ones. All these phenomena are as expected.

Runtime. Besides accuracy, we also evaluate the computation efficiency of our method against others. Since the LiTAMIN2 [24] is not open-sourced, we compare our efficiency with MULLS, Suma, FAST-LIO2 and Lego-Loam. For Suma and Lego-loam, we use their own implementation on Github with the default parameters. Similar to the accuracy evaluation, all loop-closure modules of MULLS, Suma and Lego-Loam are completely turned off to avoid unnecessary computations. Suma uses GPUs to accelerate, while other methods only use CPU. The statistics of time consumption per LiDAR scan averaged over all KITTI training sequences are summarized in Table III. Suma takes more time in our experiment than in the original paper [14] due to the difference in GPU performance. As can be seen, our method has the lowest processing time, and the next is FAST-LIO2 [13]. When compared to MULLS [30], we do not need feature extraction and save the time therein. When compared to FAST-LIO2 [13] and Lego-Loam [7], we use the voxel map described in III-B2 to avoid building a kd-tree in real-time [13] or periodically [7]. Moreover, **searching neighbor points on kd-trees has a time complexity $O(m \log(n))$** , where m is the dimension and n is the number of points in the dense point cloud map. In contrast, **the speed of searching adjacent planes in the voxel map is near $O(1)$** , which ensures efficiency.

Approach	Ours (full)	MULLS [30]	Suma [14]	FAST-LIO2 [13]	Lego-Loam [7]
mean (ms)	26.45	67.82	129.36	29.11	57.13
std (ms)	9.87	27.52	62.22	10.94	21.52

TABLE III: Comparison of Time Consumption per Scan

B. Indoor Environment Test

In this experiment, we test our method in indoor environments with a solid-state LiDAR Intel L515. As in experiment

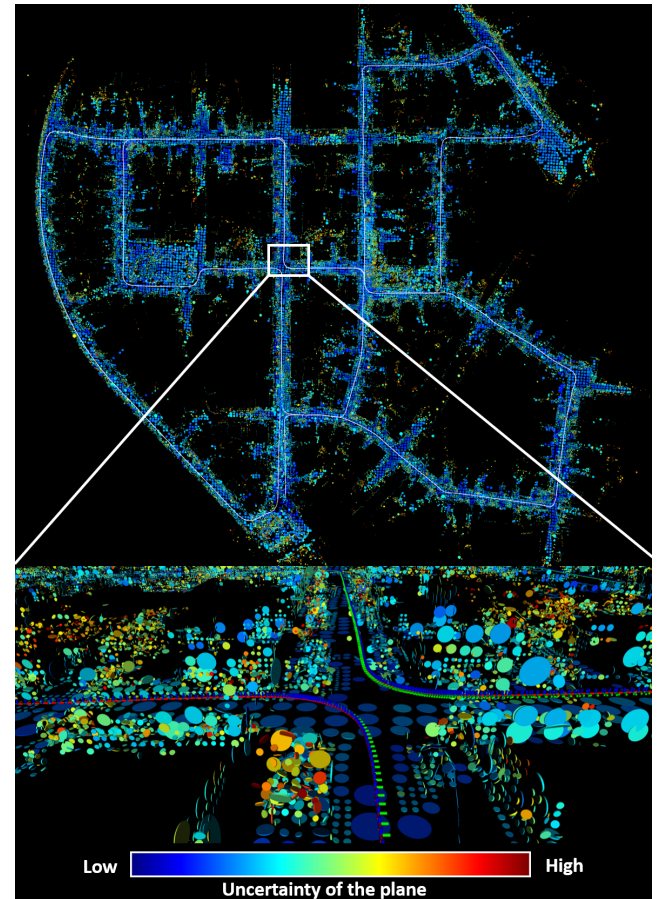


Fig. 6: Illustration of plane features in our adaptive voxel maps on KITTI training sequence 00. The plane feature is colored by its uncertainty (the trace of the plane covariance matrix in equation (8)). The plane is displayed as a disk with radius determined by the maximum eigen value of the points covariance matrix $\mathbf{A}$ defined in equation (4).

$\mathbf{A}$, a constant velocity model is used to provide the motion prior. We use a handheld device to collect three groups of LiDAR data in laboratory and warehouse environments. All data collection routes start and end intentionally at the same place. We compare our method with SSL-SLAM [33], a LiDAR SLAM approach designed for this type of LiDAR. We

run our algorithm on all datasets with the same parameters, where the maximum voxel size is 0.5m and the maximum octo-tree layer is 2. For SSL_SLAM, we conduct the experiments using the open-sourced implementation with the default parameters. Table IV records the end to end error and average calculation time per scan of our method and SSL_SLAM. Compared with SSL_SLAM, our method avoids feature extraction and considers the uncertainty of both the LiDAR point and plane map, resulting in better accuracy and efficiency than SSL_SLAM. We also test the two methods on the dataset used by the original SSL_SLAM paper [33], and the results are comparable (0.12m / 5.1ms for our method and 0.15m / 25.1ms for SSL_SLAM [33]). The small improvement is mainly attributed to the less challenging slow motion in the data, which was collected on a UGV moving quite stably and slowly. In contrast, the three sequences in this paper are collected on hands with much faster motions (especially rotations). Through the comparison, we see that our method is more robust to motion speed.

Method	End-to-End error (m) / Avg. Comp. Time (ms)		
	Lab1 (25.9m)	Lab2 (32.6m)	Warehouse (36.2m)
Ours (full)	0.02/7.18	0.02/7.26	0.01/6.38
SSL_SLAM [33]	0.71/27.7	5.80/32.9	0.93/28.3

TABLE IV: Comparison Results on L515 Datasets

We further show the aggregated colored point cloud map of our method and SSL_SLAM in the warehouse environment in Fig. 1 of the supplementary material [34]. It can be seen that our colored point cloud map is more precision, while SSL_SLAM's is more blurred, especially for the area near the return route.

C. Unstructured Environment Test

Both experiments in Section IV-A and Section IV-B are carried out in structural environments. To further verify the adaptability and applicability of our method in unstructured environments, we conduct experiments in a park and a mountain environment using a non-repetitive scanning solid-state LiDAR Livox Avia. The sensor has a built-in IMU and together they provide 10 Hz LiDAR data and 200 Hz IMU data. We use the built-in IMU to compensate the motion distortion and provide motion prior similar to FAST-LIO2 [13]. In these experiments, the same computation platform and parameters (the maximum voxel size is 3m and the maximum octo-tree layer is 3) as in Section IV-A are used despite the drastically different LiDAR scanning patterns and environment structures. In order to validate the advantages of our map structure, we compare our results with Faster-LIO [35] and FAST-LIO2 [13]. The three methods use the same IKFoM framework [32] and plane features, except for the map structure. Specifically, our method uses a probabilistic adaptive voxel map, Faster-LIO uses a Hash-indexed iVoxel map, and FAST-LIO2 uses an incremental k-d tree. For a fair comparison, the default 1:3 downsample used in Faster-LIO and FAST-LIO2 are turned off, so all raw points are used in all three methods. The other settings are kept default as in their open release. All results are averaged over 5 runs.

1) *Park Environment*: In this experiment, we use a hand-held device to collect the LiDAR and IMU data in a large park filled with trees. In order to show the drift, the data collection route starts and ends at the same place. Besides, to avoid randomness, we collect the data of three different trajectories, of which two groups are 485m long around the periphery of the park, and one group is 815m long around the periphery and the interior of the park. The mapping result of the park with the longest trajectory is shown in Fig. 2 of the supplementary material [34].

2) *Mountain Environment*: In this experiment, we use a UAV to carry our LiDAR and face the LiDAR to the ground to collect point cloud data on a mountain. Compared to the park, the mountain environment is even less structured and more challenging. The final mapping results and some partially enlarged views can be seen in Fig. 3 of the supplementary material [34]. We can clearly see the large planes with low uncertainty in the highway between mountains and small planes with high uncertainties on vegetation areas.

3) *Result Analysis*: The end to end error and average computation time per scan of the three methods under comparison are shown in Table V. As can be seen, our method achieves much higher accuracy than both Faster-LIO and FAST-LIO2 in all three sequences, mainly due to the accurate consideration of plane uncertainty in the map where Faster-LIO and FAST-LIO2 both assume deterministic plane maps. In terms of computation efficiency, both our method and Faster-LIO use a Hash table to organize the map, so they both outperform FAST-LIO2 which uses a variant kd-tree as explained in Section IV-A. When compare among our method and Faster-LIO, our method requires an additional calculation of the probability distribution of each point and the plane. While this additional calculation requires more computation time, the computed probabilistic information enables to reject further outliers as detailed in Section III-C. So the computation time of our method and Faster-LIO is more or less comparable (see Table V).

Method	End-to-End error (m) / Avg. Comp. Time (ms)			
	Park1 (485m)	Park2 (485m)	Park3 (815m)	Mountain (3490m)
Ours (full)	0.03/8.1	0.02/8.3	0.01/7.7	6.9/34.2
Faster-LIO [35]	0.88/9.8	0.99/9.8	0.07/9.9	9.8/32.0
FAST-LIO2 [13]	1.09/12.9	0.76/13.4	0.08/12.9	7.1/44.3

TABLE V: Comparison Results on Livox Avia Datasets

V. CONCLUSION

This paper proposes an efficient, probabilistic adaptive voxel mapping method for online LiDAR odometry. An accurate uncertainty model of the LiDAR point and plane is proposed by considering both LiDAR measurements noises and sensor pose estimation errors. The voxel map is constructed using coarse-to-fine adaptive resolution method, which is robust to different LiDAR scanning patterns and environments. This paper also shows how to implement the proposed mapping method in an iterated extended Kalman filter-based LiDAR(-inertial) odometry. The tests on the KITTI dataset show that our method can achieve better performance than all point cloud-based, NDT-based, and surfel-based methods. The proposed voxel mapping method not only performs well in the

spinning LiDAR in structured urban environments, but also shows good performance with solid-state LiDAR L515 in indoor environments and non-repetitive small FoV LiDAR in unstructured environments such as parks and mountains. A direction to improve this work is to add additional features such as edge to the voxel to expand the universality of the proposed map representation method.

ACKNOWLEDGMENT

This project is supported by DJI under the grant 200009538 and in part by SUSTech startup fund (Y01966105). The authors gratefully acknowledge Livox Technology for the equipment support during the whole work. The authors would like to thank Ambit-Geospatial and Jiarong Lin for the helps in the outdoor aerial experiment and indoor experiment.

REFERENCES

- [1] Z. Liu, F. Zhang, and X. Hong, "Low-cost retina-like robotic lidars based on incommensurable scanning," *IEEE/ASME Transactions on Mechatronics*, pp. 1–1, 2021.
- [2] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? the kitti vision benchmark suite," in *2012 IEEE conference on computer vision and pattern recognition*. IEEE, 2012, pp. 3354–3361.
- [3] W. Xu and F. Zhang, "Fast-lio: A fast, robust lidar-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, pp. 1–1, 2021.
- [4] Y. Okubo, C. Ye, and J. Borenstein, "Characterization of the hokuyo urg-04lx laser rangefinder for mobile robot obstacle negotiation," in *Unmanned Systems Technology XI*, vol. 7332. International Society for Optics and Photonics, 2009, p. 733212.
- [5] R. Ravi, A. Habib, and D. Bullock, "Pothole mapping and patching quantity estimates using lidar-based mobile mapping systems," *Transportation Research Record*, vol. 2674, no. 9, pp. 124–134, 2020.
- [6] J. Zhang and S. Singh, "Loam: Lidar odometry and mapping in real-time," in *Robotics: Science and Systems*, vol. 2, no. 9, 2014.
- [7] T. Shan and B. Englot, "Lego-loam: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 4758–4765.
- [8] C. Qin, H. Ye, C. E. Pranata, J. Han, S. Zhang, and M. Liu, "Lins: A lidar-inertial state estimator for robust and efficient navigation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 8899–8906.
- [9] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and R. Daniela, "Lio-sam: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2020, pp. 5135–5142.
- [10] K. Li, M. Li, and U. D. Hanebeck, "Towards high-performance solid-state-lidar-inertial odometry and mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5167–5174, 2021.
- [11] G. C. Sharp, S. W. Lee, and D. K. Wehe, "Icp registration using invariant features," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 1, pp. 90–102, 2002.
- [12] J. Servos and S. L. Waslander, "Multi channel generalized-icp," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 3644–3649.
- [13] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "Fast-lio2: Fast direct lidar-inertial odometry," *IEEE Transactions on Robotics*, pp. 1–21, 2022.
- [14] J. Behley and C. Stachniss, "Efficient surfel-based slam using 3d laser range data in urban environments," in *Proc. of Robotics: Science and Systems (RSS)*, 2018.
- [15] X. Chen, A. Milioto, E. Palazzolo, P. Giguere, J. Behley, and C. Stachniss, "Suma++: Efficient lidar-based semantic slam," in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2019, pp. 4530–4537.
- [16] D. Wisth, M. Camurri, S. Das, and M. Fallon, "Unified multi-modal landmark tracking for tightly coupled lidar-visual-inertial odometry," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1004–1011, 2021.
- [17] D. Droschel, M. Schwarz, and S. Behnke, "Continuous mapping and localization for autonomous navigation in rough terrain using a 3d laser scanner," *Robotics and Autonomous Systems*, vol. 88, pp. 104–115, 2017.
- [18] J. Quenzel and S. Behnke, "Real-time multi-adaptive-resolution-surfel 6d lidar odometry using continuous-time trajectory optimization," *arXiv preprint arXiv:2105.02010*, 2021.
- [19] J. Stückler and S. Behnke, "Multi-resolution surfel maps for efficient dense 3d modeling and tracking," *Journal of Visual Communication and Image Representation*, vol. 25, no. 1, pp. 137–147, 2014.
- [20] C. Park, S. Kim, P. Moghadam, C. Fookes, and S. Sridharan, "Probabilistic surfel fusion for dense lidar mapping," in *Proceedings of the IEEE International Conference on Computer Vision Workshops*, 2017, pp. 2418–2426.
- [21] C. Park, P. Moghadam, J. L. Williams, S. Kim, S. Sridharan, and C. Fookes, "Elasticity meets continuous-time: Map-centric dense 3d lidar slam," *IEEE Transactions on Robotics*, 2021.
- [22] M. Magnusson, A. Lilienthal, and T. Duckett, "Scan registration for autonomous mining vehicles using 3d-ndt," *Journal of Field Robotics*, vol. 24, no. 10, pp. 803–827, 2007.
- [23] J. Saarinen, H. Andreasson, T. Stoyanov, J. Ala-Luhtala, and A. J. Lilienthal, "Normal distributions transform occupancy maps: Application to large-scale online 3d mapping," in *2013 IEEE international conference on robotics and automation*. IEEE, 2013, pp. 2233–2238.
- [24] M. Yokozuka, K. Koide, S. Oishi, and A. Banno, "Litamin2: Ultra light lidar-based slam using geometric approximation applied with kl-divergence," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 11 619–11 625.
- [25] M. Tatarchenko, A. Dosovitskiy, and T. Brox, "Octree generating networks: Efficient convolutional architectures for high-resolution 3d outputs," in *Proceedings of the IEEE international conference on computer vision*, 2017, pp. 2088–2096.
- [26] C. Häne, S. Tulsiani, and J. Malik, "Hierarchical surface prediction for 3d object reconstruction," in *2017 International Conference on 3D Vision (3DV)*. IEEE, 2017, pp. 412–420.
- [27] C. Yuan, X. Liu, X. Hong, and F. Zhang, "Pixel-level extrinsic self calibration of high resolution lidar and camera in targetless environments," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7517–7524, 2021.
- [28] Z. Liu and F. Zhang, "Balm: Bundle adjustment for lidar mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3184–3191, 2021.
- [29] J.-E. Deschaud, "Imls-slam: Scan-to-model matching based on 3d data," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 2480–2485.
- [30] Y. Pan, P. Xiao, Y. He, Z. Shao, and Z. Li, "Mulls: Versatile lidar slam via multi-metric linear least square," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 11 633–11 640.
- [31] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual-(inertial) odometry," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 7244–7251.
- [32] D. He, W. Xu, and F. Zhang, "Kalman filters on differentiable manifolds," 2021. [Online]. Available: <https://arxiv.org/abs/2102.03804>
- [33] H. Wang, C. Wang, and L. Xie, "Lightweight 3-d localization and mapping for solid-state lidar," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1801–1807, 2021.
- [34] C. Yuan, W. Xu, X. Liu, X. Hong, and F. Zhang, "Supplementary material: Efficient and probabilistic adaptive voxel mapping for accurate online lidar odometry," https://github.com/hku-mars/VoxelMap/blob/master/supply/VoxelMap_Supplementary_material.pdf.
- [35] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-lio: Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4861–4868, 2022.